Phase 1: Environment & Data Preparation

Before running the code, ensure your folder structure is initialized and you have "seed" data for the model to process.

1. **Install Dependencies**:

Open your terminal in the project root and run:

pip install flask torch torchvision opencv-python pillow matplotlib seaborn scikit-learn

2. **Initialize Folders**:

Ensure the following directory structure exists:

Plaintext

/static/uploads/

/static/results/

/data/labeled/

/data/unlabeled/

/models/

3. **Generate Test Data**:

Run your setup_data.py script. This creates dummy images so the training pipeline doesn't crash due to empty folders.

Terminal

python setup_data.py

Phase 2: The Training Pipeline (Algorithms 4 & 5)

This is where the **SimCLR** and **Semi-Supervised** logic happens.

1. **Run the Pipeline**:

Execute python train_pipeline.py.

2. **What happens internally?**
    - **SimCLR Pre-training**: The model looks at images in /data/unlabeled/ and learns to group similar retinal features.
    - **Fine-tuning**: The model uses the small set in /data/labeled/ to learn the 5 DR classes.

- **Pseudo-labeling**: The model makes high-confidence guesses on the remaining unlabeled data to improve itself.

3. **Output**: You will see a file named dr_model_final.pth appear in the /models/ folder. **Do not proceed until this file exists.**

Phase 3: Analytics Generation

To populate the dashboard with the "Reports" you requested, you must generate the performance graphs.

1. **Execute Reporter**:

Terminal

python generate_reports.py

2. **Verification**: Check static/results/. You should now see training_report.png, confusion_matrix.png, and tsne_clusters.png.

Phase 4: Launching the Dashboard (Inference & XAI)

Now that the model is "smart" and the reports are ready, launch the web interface.

1. **Start Flask**:

Terminal

python app.py

2. **Access the UI**:

Open your web browser and go to: http://127.0.0.1:5000

Phase 5: Using the System

1. **Upload**: Select a retinal fundus image (from your data folder or a sample image).
2. **Analyze**: Click **"Start AI Analysis"**.
3. **Interpret Output**:
    - **Classification**: View the DR grade (e.g., "Moderate DR").
    - **Confidence**: Check the percentage bar (e.g., "94.5%").
    - **Grad-CAM Heatmap**: Look at the colored overlay on the eye image. Red spots indicate where the model detected hemorrhages or lesions.

Summary of Script Execution Order:

Step	Script	Purpose
1	setup_data.py	Creates folders and dummy images.
2	train_pipeline.py	Executes SimCLR and Pseudo-labeling.
3	generate_reports.py	Generates the performance graphs (Analytics).
4	app.py	Starts the Explainable AI Dashboard.